

Explanation Report: Academic Advisor Expert System

APT 3020 - Lab 2 Submission

1. Introduction & Overview

This is my explanation report for the Intelligent Student Academic Advisor system. Basically, it's a rule-based expert system I built to check a student's stats—like their GPA, attendance, disciplinary record, and fees—and automatically figure out their academic standing (like if they qualify for a scholarship or if their registration should be blocked).

2. Knowledge Representation

I made sure to separate the core logic (the Python inference engine) from the actual rules (the knowledge base). I used JSON for the knowledge base because it's clean, easy to read, and means we can update the rules later without touching the code.

Generating Facts

Instead of doing messy math inside the rules, the system first takes the raw inputs (e.g., GPA = 3.8) and translates them into simple True/False facts (e.g., "GPA > 3.5"). This makes the rule-checking part way simpler.

Rule Setup

Each rule in the JSON file is basically an object with a Name, a list of Conditions (the "IF" part), and a Conclusion (the "THEN" part).

3. Reasoning Engine & Inference Strategy

For the reasoning engine, I went with Forward Chaining. Since we already have the student's data right from the start, a data-driven approach makes the most sense.

- The engine loads up all the facts we generated about the student.
- It loops through the JSON Knowledge Base.
- For each rule, it checks if all the required conditions are currently in our list of facts.
- If they match, the rule "fires" and we save the conclusion.

- It doesn't stop after finding one match. It checks everything so a student can get multiple outcomes (like qualifying for Graduation AND the Dean's List at the same time).

4. Testing the Inference Engine (The 3 Profiles)

To show that the forward chaining actually works, I ran three different types of students through the system:

Profile 1: The High Achiever

Raw Input: GPA = 3.8, Attendance = 90%, No Disciplinary Cases, Prerequisites Done, No Outstanding Fees.

Generated Facts: "GPA > 3.5", "GPA > 3.0", "Attendance > 80%", "No Disciplinary Cases", "Completed Prerequisite Courses", "No Outstanding Fees".

Rules Triggered: Scholarship Rule, Graduation Rule, Dean's List Rule.

Conclusions:

- Eligible for Scholarship
- Eligible for Graduation
- Dean's List Candidate

Profile 2: The Struggling Student

Raw Input: GPA = 2.5, Attendance = 75%, Has Disciplinary Cases, Prerequisites Not Done, Owes Fees.

Generated Facts: "GPA < 3.0", "Attendance Below 80%", "Has Disciplinary Cases", "Outstanding Fees".

Rules Triggered: Probation Rule, Registration Rule.

Conclusions:

- Academic Probation
- Registration Blocked

Profile 3: The Average Student (Blocked by Fees)

Raw Input: GPA = 3.2, Attendance = 85%, No Disciplinary Cases, Prerequisites Done, Owes Fees.

Generated Facts: "GPA Between 3.0 and 3.49", "GPA > 3.0", "Attendance > 80%", "No Disciplinary Cases", "Completed Prerequisite Courses", "Outstanding Fees".

Rules Triggered: Registration Rule.

Conclusions:

- Registration Blocked

Note: Even though they had the GPA and classes to graduate, the Graduation rule didn't fire because the "No Outstanding Fees" fact was missing.

5. Explanation Facility

A good expert system needs to explain itself, so I added a feature that shows exactly why a conclusion was reached by printing out the matched conditions.

Example of what the system outputs:

✓ Eligible for Scholarship because:

- GPA > 3.5
- Attendance > 80%
- No Disciplinary Cases

Therefore, the Scholarship Rule was activated.